

VELLORE INSTITUTE OF TECHNOLOGY
CHENNAI

School of Computer Science Engineering and Information Systems
Academic Year 2026 – 2027

SOFTWARE REQUIREMENTS
SPECIFICATION

Initial Version — Review 1

ServiceDesk AI

A Cloud-Native Customer Support SaaS Platform for Small Businesses

Project 19 · Customer Support SaaS

“Manage customer complaints and automate ticket handling.”

PREPARED BY

NAME	REGISTRATION NUMBER
Aarush Jagannathan	23BLC1211
Gomatheswar M	23BLC1195

B.Tech. Electronics and Computer Engineering

UNDER THE GUIDANCE OF

Dr. M Dinakaran

BCSE408L — CLOUD COMPUTING

AUGUST 2026

Document Control

This is the initial baseline of the Software Requirements Specification, prepared for Review 1. It is expected to be revised as design work proceeds through Reviews 2 and 3.

Field	Detail
Document title	Software Requirements Specification — ServiceDesk AI
Version	1.0 (Initial)
Status	Baseline for Review 1
Prepared by	Aarush Jagannathan (23BLC1211), Gomatheswar M (23BLC1195)
Guide	Dr. M Dinakaran
Course	BCSE408L — Cloud Computing
Institution	School of Computer Science Engineering and Information Systems, VIT Chennai
Project	Project 19 — ServiceDesk AI, Customer Support SaaS
Source of record	docs/SRS.md in the project repository

Revision History

Version	Date	Author	Description of change
1.0	August 2026	Both members	Initial baseline prepared for Review 1

Purpose of this document

This specification states what the ServiceDesk AI system must do, for whom, and under what constraints. It deliberately does not describe how the system will be implemented; architecture, database design and interface design are covered in separate documents produced for Review 2.

Requirements are drawn from three sources: the BCSE408L Cloud-Native SaaS Project Guidelines, the Project Review Schedule and Report Template, and the Project 19 entry in the Cloud SaaS Project Requirements. Where this specification adds a requirement not present in those documents, it is marked explicitly as a design decision rather than presented as an official requirement.

Table of Contents

Select the table below and press F9 in Microsoft Word to populate the page numbers.

ServiceDesk AI — A Cloud-Native Customer Support SaaS Platform for Small Businesses

Course	BCSE408L — Cloud Computing
Institution	School of Computer Science Engineering and Information Systems, VIT Chennai
Academic year	2026–2027
Project	Project 19 — ServiceDesk AI
SaaS category	Customer Support SaaS
Team	Aarush Jagannathan (23BLC1211), Gomatheswar M (23BLC1195)
Guide	Dr. M Dinakaran

1. Introduction

1.1 Purpose

This document specifies the requirements for ServiceDesk AI, a cloud-native Software-as-a-Service application that lets a small business receive customer complaints, convert them into trackable support tickets, route them to support agents, and measure how well it is responding.

It is the reference against which the implementation is built and evaluated, and it covers the functional and non-functional requirements defined in the official BCSE408L project specification for Project 19.

1.2 Scope of this document

The document describes what the system does, who uses it, and the constraints it operates under. Design and implementation detail is covered separately in `SYSTEM_ARCHITECTURE.md`, `CLOUD_ARCHITECTURE.md`, `DATABASE_DESIGN.md` and `API_DOCUMENTATION.md`.

1.3 Definitions

Term	Meaning
Ticket	A single customer complaint or support request, tracked from submission to closure
Reference	The human-readable ticket identifier, e.g. SD-1024
Lifecycle	The permitted sequence of ticket statuses
RBAC	Role-Based Access Control
Internal note	A message on a ticket visible to staff only
Fallback classifier	The deterministic rule-based engine used when the AI provider is unavailable
SaaS	Software as a Service — software delivered over the internet as a multi-user, centrally hosted service

1.4 References

1. BCSE408L Cloud-Native SaaS Project Guidelines, VIT Chennai, AY 2026–2027
2. BCSE408L Project Review Schedule & Report Template
3. BCSE408L Cloud SaaS Project Requirements — Project 19: ServiceDesk AI

2. Problem Statement

"Manage customer complaints and automate ticket handling."

— Official problem statement, BCSE408L Project List, Project 19

A small business receiving customer complaints across email, messaging apps and phone calls has no reliable record of what was reported, who is handling it, or whether it was ever resolved.

Requirement analysis identified four concrete failures behind that one-line statement:

4. **No single intake point.** Requests arrive in several channels and are never consolidated.
5. **No ownership.** Without an assigned owner, a complaint is answered twice by different people or not at all.
6. **No status or history.** The customer cannot check progress, so they follow up repeatedly, generating more work.
7. **No measurement.** The business cannot say how many issues arrived, in what categories, or how long they took.

3. Existing System

Three approaches are currently in common use, each with clear limitations.

3.1 Shared email inbox

- No ticket identity or reference number
- No ownership — replies collide
- No status beyond read/unread
- History scattered across separate threads
- No reporting of any kind

3.2 Spreadsheet tracker

- Updated by hand, so it goes stale immediately
- No customer-facing view at all
- No access control — every user can edit every row
- Breaks down beyond a few dozen rows
- Not accessible outside the office

3.3 Commercial helpdesk products (Zendesk, Freshdesk, Jira Service Management)

- Priced per agent, per month
- Assume a dedicated support team
- Require substantial configuration before delivering value
- Provide far more capability than a small team needs, at a cost that scales faster than the business

The gap: a small business needs helpdesk structure without helpdesk pricing or helpdesk complexity.

4. Proposed System

ServiceDesk AI is a multi-user, cloud-hosted SaaS application. Every complaint becomes a ticket carrying an owner, a category, a priority and a tracked status — visible to the customer, actionable by the agent, and measurable by the administrator.

Workflow

8. A customer raises a ticket with a subject, description and optional category.
9. The system classifies it — proposing category, priority and sentiment, and generating a one-line summary.
10. The ticket enters the agent queue, where it can be assigned, prioritised, filtered and searched.
11. An agent resolves it, optionally starting from an AI-generated draft reply that they review and edit.
12. The business measures everything through dashboards and exportable reports built on live data.

What makes it cloud-native

- The database is a managed cloud service with its own backups and availability guarantees.
- The REST API is stateless — all state lives in the database — so the platform can run multiple replicas.
- The frontend is a static build distributed by a global CDN.
- The application is reachable from any browser on the public internet and never depends on localhost.

5. Objectives

The five objectives below are quoted from the official Project 19 specification, with the means of satisfying each.

Objective	How it is met
Develop a scalable cloud-native SaaS application	Three independently deployable tiers: React SPA, Express REST API, managed PostgreSQL
Implement secure role-based authentication	JWT sessions, bcrypt-hashed passwords, roles enforced in the API
Provide dashboards and analytical reports	Role-specific dashboards and CSV-exportable reports computed from live ticket data
Deploy the application on a public cloud platform	Vercel, Render and Supabase, reachable at a public URL
Demonstrate collaborative software engineering practices	One team repository, regular commits, documented design, automated tests

6. Scope

6.1 In scope

- Registration, login and profile management for three roles
- Role-based access control enforced in the REST API
- Full ticket lifecycle: create, read, update, delete, assign and status transitions
- Threaded customer-agent conversation with staff-only internal notes
- Search, filtering and pagination across all ticket fields
- Role-specific dashboards and exportable analytical reports
- Persistent in-app notifications
- AI classification, summarisation, suggested resolution steps and agent-reviewed draft replies
- Administration: users, agents, roles, categories and an activity audit log
- Responsive interface for desktop, tablet and mobile
- Public cloud deployment with a live URL

6.2 Out of scope

Deliberately excluded to keep the delivered system complete and reliable rather than broad and unfinished:

- Payment or subscription billing
- Live chat and telephony integration
- Native mobile applications
- Multi-language support
- Third-party CRM integrations
- An automated SLA escalation engine
- Fully autonomous AI replies sent without agent approval

7. Target Users

The specification names three actors.

Actor	Description
Customer	A person who has bought from the business and needs help. Raises tickets and follows their progress.
Support Agent	A staff member who resolves tickets. Works from a prioritised queue across all customers.
Admin	The business owner or support manager. Manages users, agents, categories and configuration, and monitors overall performance.

8. Functional Requirements

The ten requirements below are the official Project 19 functional requirements. Each is expanded into the specific behaviour implemented.

FR-1 — User registration and login

- FR-1.1 A visitor can register with full name, email address and password.
- FR-1.2 Email addresses are unique; a duplicate registration is rejected.
- FR-1.3 Passwords must be at least 8 characters and contain a letter and a number.
- FR-1.4 Passwords are stored only as bcrypt hashes.
- FR-1.5 A registered user can sign in and receive an authenticated session.
- FR-1.6 Sessions can be refreshed without re-entering credentials, and ended by signing out.
- FR-1.7 Self-registration always creates a Customer; privileged roles are granted only by an Admin.

FR-2 — Profile management

- FR-2.1 A user can view their profile.
- FR-2.2 A user can update their full name and phone number.
- FR-2.3 A user can change their password by supplying the current one.
- FR-2.4 A user cannot change their own role or email address.

FR-3 — Role-based access control

- FR-3.1 Three roles exist: Customer, Support Agent, Admin.
- FR-3.2 Every protected endpoint verifies the caller's role.
- FR-3.3 Ticket endpoints additionally verify record ownership.
- FR-3.4 A Customer cannot access another customer's ticket, any admin function, or any agent-only control.
- FR-3.5 An Agent cannot manage users, roles or categories, and cannot delete tickets.
- FR-3.6 Access control is enforced in the API independently of the user interface.
- FR-3.7 A deactivated account is refused access immediately, even with an unexpired token.

FR-4 — CRUD operations

- FR-4.1 A Customer can create a ticket with subject, description and optional category and priority.
- FR-4.2 Any permitted user can read a ticket and its conversation.
- FR-4.3 Staff can update a ticket's subject, priority, category and assignment.
- FR-4.4 Only an Admin can delete a ticket; deletion cascades to its messages and events.
- FR-4.5 Ticket status changes follow the lifecycle in §8.11; invalid transitions are rejected.

- FR-4.6 An Admin can create, update and deactivate users, and create, update and delete categories.

FR-5 — Search and filtering

- FR-5.1 Tickets can be filtered by status, priority, category, assigned agent, sentiment and date range.
- FR-5.2 Tickets can be searched by reference, subject, customer name and body keywords.
- FR-5.3 Results are paginated with a reported total.
- FR-5.4 Results are sortable by creation date, update date, priority and status.
- FR-5.5 A Customer's search is scoped to their own tickets regardless of the filters supplied.

FR-6 — Dashboard

- FR-6.1 A Customer sees total, open and resolved ticket counts, tickets awaiting their reply, and recent tickets.
- FR-6.2 An Agent sees assigned, open, high-priority and resolved counts, average resolution time, category distribution and their prioritised queue.
- FR-6.3 An Admin sees totals, unassigned count, active users, average resolution time, and distributions by status, priority, category and sentiment, plus agent workload, a 30-day volume trend and AI usage statistics.
- FR-6.4 Every dashboard figure is computed from the database at request time. No figure is hard-coded.

FR-7 — Notifications

- FR-7.1 A persistent in-app notification is created when a ticket is assigned, replied to, or has its status changed.
- FR-7.2 A user sees only their own notifications.
- FR-7.3 An unread count is displayed in the application shell.
- FR-7.4 Notifications can be marked read individually or all at once.

FR-8 — Report generation

- FR-8.1 Reports are available for tickets by status, priority and category; sentiment distribution; agent workload; created-versus-resolved trend; and average resolution time by category.
- FR-8.2 Reports are restricted to Agents and Admins.
- FR-8.3 Ticket data can be exported as a CSV file.

FR-9 — Cloud database integration

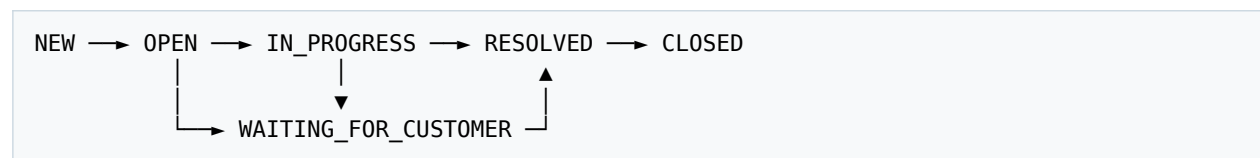
- FR-9.1 All persistent data is stored in a managed cloud PostgreSQL instance.
- FR-9.2 The schema is applied by versioned SQL migrations.

- FR-9.3 The frontend never connects to the database; it communicates only through the REST API.
- FR-9.4 Referential integrity and business invariants are enforced by database constraints as well as application code.

FR-10 — Administrator module

- FR-10.1 An Admin can list, search and filter all users.
- FR-10.2 An Admin can create user accounts with any role.
- FR-10.3 An Admin can change a user's role and deactivate or reactivate an account.
- FR-10.4 An Admin cannot remove their own admin role or deactivate their own account.
- FR-10.5 An Admin can manage ticket categories.
- FR-10.6 An Admin can view an activity audit log of every privileged action.

8.11 Ticket lifecycle



From	Permitted next states
NEW	OPEN, IN_PROGRESS, CLOSED
OPEN	IN_PROGRESS, WAITING_FOR_CUSTOMER, RESOLVED, CLOSED
IN_PROGRESS	WAITING_FOR_CUSTOMER, RESOLVED, CLOSED
WAITING_FOR_CUSTOMER	IN_PROGRESS, RESOLVED, CLOSED
RESOLVED	CLOSED, OPEN (reopen)
CLOSED	OPEN (Admin only)

A Customer may only close a resolved ticket or reopen it if the issue persists. All other transitions are staff actions.

8.12 AI assistance *(design decision, not a specification requirement)*

The official Project 19 specification does not require AI; only the project title contains it. The following capabilities are our own addition, and the system is built so that **no requirement above depends on them**.

- **AI-1** On submission, the system proposes a category, priority and sentiment, and generates a one-line summary.
- **AI-2** An agent can generate a draft reply, which they must review and edit before sending.

- **AI-3** An agent can generate a summary of a long conversation.
- **AI-4** An agent can request suggested troubleshooting steps.
- **AI-5** The AI never sends a message to a customer. Only an agent action creates a customer-visible message.
- **AI-6** If the AI provider is unreachable, a deterministic rule-based classifier produces the result instead, and the interface says so.
- **AI-7** Classification runs after the ticket is committed, so ticket creation cannot fail because of the AI service.
- **AI-8** The AI provider is called only from the backend; the API key is never exposed to the browser.

9. Non-Functional Requirements

The six quality attributes required by the specification.

NFR-1 — Security

- Passwords hashed with bcrypt at cost factor 12; never stored or logged in plaintext.
- Stateless JWT access tokens (15 minutes) with refresh tokens in `httpOnly` cookies.
- Every protected route checks role; every ticket handler checks ownership.
- All request bodies and query strings validated with schemas before use.
- All database access through parameterised queries — no string-built SQL.
- Secrets supplied only through environment variables; `.env` is never committed.
- CORS restricted to an explicit origin allow-list; security headers set by Helmet; login and registration rate limited.
- Errors never expose stack traces or raw database messages to users.

NFR-2 — Scalability

- The API holds no session state, so the platform can run multiple replicas behind a load balancer.
- Database access uses a bounded connection pool.
- List endpoints are paginated, so response size does not grow with the dataset.
- The frontend is static and served from a CDN, independent of API load.

NFR-3 — Availability

- All three tiers are managed cloud services with platform-level uptime.
- A health endpoint reports both process and database health.
- The AI feature degrades to the fallback classifier rather than failing.
- The API shuts down gracefully on SIGTERM so redeployments do not drop in-flight requests.

NFR-4 — Performance

- Indexes on every column used for filtering, plus a GIN full-text index for keyword search.
- Server-side pagination with a maximum page size of 100.
- Client-side query caching to avoid redundant requests.
- The AI call is off the critical path of ticket creation.

NFR-5 — Reliability

- Business invariants enforced by database constraints as well as application code.
- Lifecycle transitions validated server-side from a single transition table.
- Related writes wrapped in transactions.
- Every privileged action recorded in an audit log; every ticket change recorded on its timeline.

NFR-6 — Usability

- One consistent design system across every screen.
- Responsive from 375 px upward: the sidebar collapses to a menu, tables become stacked cards.
- Explicit loading, empty and error states on every data view.
- Plain-language error messages; validation errors shown against the field concerned.
- Keyboard-accessible controls with a visible focus indicator.

10. User Roles and Permissions

Capability	Customer	Agent	Admin
Register and manage own profile	✓	✓	✓
Create a ticket	✓	—	✓
View own tickets	✓	✓	✓
View all tickets	—	✓	✓
Reply on a ticket	✓	✓	✓
Add an internal note	—	✓	✓
Change status, priority, category	—	✓	✓
Assign or escalate a ticket	—	✓	✓
Use AI assistance	—	✓	✓
Delete a ticket	—	—	✓
Manage users, agents and roles	—	—	✓
Manage categories	—	—	✓
View global dashboard and reports	—	✓ (reports)	✓
View audit log	—	—	✓

11. Use Cases

11.1 Actors and use cases

Customer — Register/Login · Manage Profile · Create Ticket · View Ticket · Reply to Ticket · Track Ticket Status · Receive Notifications

Support Agent — Login · View Ticket Queue · Search Tickets · Update Ticket · Assign/Escalate Ticket · Reply to Customer · Use AI Assistance · View Dashboard

Admin — Manage Users · Manage Agents · Manage Tickets · Manage Categories · View Reports · View Dashboard · Monitor Activity Log

The use case diagram is in [\[../diagrams/use-case-diagram.md\]\(../diagrams/use-case-diagram.md\)](#).

11.2 UC-01 Create Ticket

Actor	Customer
Precondition	The customer is signed in
Main flow	1. Customer opens *New ticket*. 2. Enters subject and description; optionally selects category and priority. 3. Submits. 4. System validates the input. 5. System stores the ticket with status NEW and assigns a reference. 6. System records a CREATED event. 7. System returns the ticket and navigates to it. 8. Classification runs in the background and updates category, priority, sentiment and summary.
Alternate flow	4a. Validation fails → field-level errors are shown and nothing is stored.
Exception flow	8a. The AI service is unavailable → the fallback classifier runs; if that also fails the ticket simply remains unclassified. The ticket itself is unaffected.
Postcondition	A ticket exists, visible to its customer and to all staff

11.3 UC-02 Reply with AI Assistance

Actor	Support Agent
Precondition	The agent is signed in and viewing a ticket that is not closed
Main flow	1. Agent opens the AI assistance panel. 2. Selects *Generate* under Draft reply. 3. System builds the conversation context and requests a draft. 4. System stores the suggestion and returns the draft. 5. Agent reads and edits the draft. 6. Agent selects *Insert into reply*. 7. Agent edits further in the reply box and sends. 8. System stores the message flagged as AI-assisted, sets the first-response timestamp, moves the ticket to IN_PROGRESS, and notifies the customer.
Alternate flow	5a. Agent discards the draft and writes their own reply.
Exception flow	3a. The AI provider fails → the fallback produces a draft and the panel states that the fallback was used.

Actor	Support Agent
Postcondition	A customer-visible reply exists, authored and sent by the agent

11.4 UC-03 Change Ticket Status

Actor	Support Agent or Admin (Customer, restricted)
Precondition	The actor can access the ticket
Main flow	1. Actor selects a target status. 2. System checks the transition against the lifecycle table and the actor's role. 3. System updates the status and the resolved/closed timestamps. 4. System records a STATUS_CHANGED event and an audit entry. 5. System notifies the customer if someone else made the change.
Exception flow	2a. The transition is not permitted → 422 with the reason, and nothing changes.
Postcondition	The ticket is in a valid state with a consistent audit trail

11.5 UC-04 Manage Users

Actor	Admin
Main flow	1. Admin opens *Users*. 2. Searches or filters by role. 3. Creates a user, changes a role, or deactivates an account. 4. System applies the change and writes an audit entry.
Exception flow	3a. The admin targets their own account for demotion or deactivation → 422, preventing lock-out.

12. System Constraints

#	Constraint
C - 1	The final application must be publicly accessible; a localhost-only deployment is not acceptable
C - 2	Deployment must use a platform permitted by the course guidelines
C - 3	The team consists of exactly two students, both of whom must be able to explain and modify the implementation
C - 4	All work must be original; copied repositories and purchased templates are prohibited
C - 5	One team repository with regular commits and a meaningful README
C - 6	Free hosting tiers impose cold starts on the API and idle suspension on the database, which must be managed before a demonstration
C - 7	No secret may be committed to version control or exposed in the browser bundle

13. Assumptions

#	Assumption
A - 1	Users have a modern browser and internet access
A - 2	A small business operates a single service desk; multi-tenancy across businesses is not modelled
A - 3	Support staff accounts are created by an administrator, not by self-registration
A - 4	English is the only interface language
A - 5	Email addresses are valid and reachable; the system does not verify them by sending mail
A - 6	The AI provider, where configured, is reachable over HTTPS; the system tolerates it not being so
A - 7	Ticket volumes are consistent with a small business — thousands, not millions, of records

14. Future Enhancements

	Enhancement	Rationale
	Email notifications in addition to in-app	Customers do not have to be signed in to learn of an update
	File attachments on tickets via cloud object storage	Screenshots make technical issues far quicker to diagnose
	Configurable SLA targets with automatic escalation	Lets the business enforce its own response commitments
	Knowledge base with AI-suggested articles	Deflects repeat questions before they become tickets
	Real-time updates over WebSockets	Removes the polling currently used for the notification badge
	Multi-tenancy so one deployment serves several businesses	The natural commercial evolution of the product
	Customer satisfaction rating after resolution	Adds a quality measure alongside speed